BARWISE SECTION BOUNDARY DETECTION IN SYMBOLIC MUSIC USING CONVOLUTIONAL NEURAL NETWORKS

Omar Eldeeb
Technical University of Munich
omar.eldeeb@tum.de

Martin E. Malandro
Sam Houston State University
malandro@shsu.edu

ABSTRACT

Current methods for Music Structure Analysis (MSA) focus primarily on audio data. While symbolic music can be synthesized into audio and analyzed using existing MSA techniques, such an approach does not exploit symbolic music's rich explicit representation of pitch, timing, and instrumentation. A key subproblem of MSA is section boundary detection—determining whether a given point in time marks the transition between musical sections. In this paper, we study automatic section boundary detection for symbolic music. First, we introduce a human-annotated MIDI dataset for section boundary detection, consisting of metadata from 6134 MIDI files that we manually curated from the Lakh MIDI dataset. Second, we train a deep learning model to classify the presence of section boundaries within a fixed-length musical window. Our data representation involves a novel encoding scheme based on synthesized overtones to encode arbitrary MIDI instrumentations into 3-channel piano rolls. Our model achieves an F_1 score of 0.77, improving over the analogous audio-based supervised learning approach and the unsupervised block-matching segmentation (CBM) audio approach by 0.22 and 0.31, respectively. We release our dataset, code, and models. [1]

1. INTRODUCTION

Music is commonly structured over time in a hierarchical manner, ranging from short repeating phrases and motifs to longer, non-overlapping sections such as verses, choruses, or movements. The automatic analysis of this structure is known as Music Structure Analysis (MSA) and can characterize analyses spanning a wide temporal range—from brief segments lasting a few seconds, to entire sections exceeding a minute in duration. A common first step in MSA is the detection of section boundaries, which can then be used to group or label segments based on principles such as homogeneity and repetition. In this work, we focus on detecting section boundaries in symbolic music in a non-hierarchical manner—that is, identifying the points in time where one musical section (e.g., verse, chorus, bridge, etc.) ends and another begins.

So far, most algorithms for MSA have focused on waveform audio as opposed to symbolic data, possibly due to a current lack of human-annotated symbolic music. Exceptions include [1], which focused on phrase-level segmentation in pop piano music (and proposed identifying sections from the patterns of detected phrases), and [2], which focused on phrase-level segmentation in melodies.

MSA of waveform audio is a central research topic in music information retrieval, and is motivated by a number of downstream applications [3]. Here we highlight two motivations for the study of MSA for symbolic music:

First, for researchers who are interested in MSA, symbolic data are more freely and openly available than waveform data. While existing annotations of waveform audio are generally freely and openly available [4–8], obtaining access to all of the associated audio recordings can be expensive. In contrast, symbolic datasets are more freely and widely available. We release a new dataset, consisting of human annotations of section boundaries of 6134 MIDI files from the Lakh MIDI Dataset (LMD) [9, 10]. All of the MIDI referenced by these annotations is available in the LMD.

Second, MSA for symbolic music has the potential to improve the quality of symbolic music generation. Previous works have identified a trend for outputs from generative symbolic models to be repetitive or meandering, rather than having clear structure that drives toward musical payoffs [11–13]. Recent work [14] introduced a structure-aware symbolic music generation system, which is capable of writing contrasting musical sections. We note that the authors of [14] used an audio-based method [15], applied to waveform data, to compute section boundaries for their training data. If MSA techniques for symbolic music outperform MSA techniques for audio, then such techniques would improve the training data quality, and therefore likely also improve the output quality, of structure-aware generative symbolic music systems.

2. RELATED WORK

Data for music structure analysis are scarce, as manually annotating music is labor-intensive. Audio datasets for

[1] Dataset available at https://github.com/m-malandro/SLMS. Code and models available at https://github.com/omareldeeb/midi-msa.

© O. Eldeeb and M. E. Malandro. Licensed under a Creative Commons Attribution 4.0 International License (CC BY 4.0). **Attribution:** O. Eldeeb and M. E. Malandro, "Barwise Section Boundary Detection in Symbolic Music Using Convolutional Neural Networks", in *Proc. of the 26th Int. Society for Music Information Retrieval Conf.*, Daejeon, South Korea, 2025.

MSA include SALAMI [4], the Harmonix dataset [5], and the RWC dataset [6–8]. Annotated symbolic datasets include the piano dataset Pop909 [16] and its annotations in [1], the Essen Folksong dataset [17] (which consists of phrase-level annotations of 8473 short folk melodies), and S3, an annotated dataset of 4 symphonies totaling 16 movements [18]. We aim to develop a method capable of segmenting arbitrary MIDI files, and therefore developed our own dataset for this work—see Section 4 for details.

In 2014, Ullrich, Schlüter, and Grill [19] express the problem of section boundary detection in audio as a binary classification task, and train a convolutional neural network (CNN) on mel-scaled spectrograms extracted from fixed-duration slices of audio to predict whether a section boundary exists at the center of a given network input. At test time, the network is applied over a sliding window of the audio and produces for each frame a boundary probability. These boundary probabilities are finally decoded to boundary positions using a simple peak-picking algorithm with a moving threshold.

In two follow-up papers [20, 21], Grill and Schlüter extend this approach by incorporating additional input representations and multi-level annotations. They use Harmonic-Percussive Source Separation (HPSS) to isolate harmonic and percussive components and Self-Similarity Lag Matrices (SSLMs) to capture long-range temporal dependencies. This approach remains the state of the art in supervised section boundary detection on the SALAMI dataset [4]. Subsequent papers have explored self-supervised learning approaches [22], hierarchical MSA [23, 24], and the functional labeling of segments [25–27]. More recently, Transformer-based models have been proposed to jointly detect boundaries and functional labels [26, 27], achieving state-of-the-art results on datasets like Harmonix [5].

3. METHOD

Our method is inspired by the audio-based method in [21] and the line of work leading up to it [19, 20]. In particular, our method involves training a convolutional neural network (CNN) on piano rolls synthesized from MIDI data.

3.1 Feature Extraction

Given a MIDI file, we extract the time, pitch, duration, velocity, and program (instrumentation) information of each note event. Additionally, we multiply the note's velocity by any preceding volume and expression control change values (scaled to the interval $[0, 1]$) on the corresponding message channel. We quantize all events to a fixed temporal resolution of 4 ticks per beat and merge all tracks—except for drums—into a single "piano roll" image. We split each piano roll into equal-size patches centered at measure boundaries, which serve as the primary inputs to our neural network. The patches have a height of 128 pixels, corresponding to the 128 MIDI pitch values, and a width of 512 pixels, corresponding to a duration of 128 quarter notes, or 32 bars in a 4/4 time signature. We emphasize that our inputs do not have to be in 4/4—we accommodate files containing any collection of time signatures, and we compute measure boundaries from the time signature information within the files. As in [21], we separate drums into a distinct channel in order to allow the network to easily distinguish between rhythmic and harmonic/melodic content. Furthermore, we disregard the duration given by drum note events and set them to an arbitrary but fixed duration of one 16th note.

3.2 Instrument Encoding

We hypothesize that explicitly encoding instrumentation (i.e., MIDI program numbers) into the piano roll representation simplifies the learning task. Given that MIDI defines a fixed number of 128 possible programs, a naive approach would be to assign each program its own input channel. However, in most cases this results in extremely sparse tensors, as most musical pieces use only a small number of instruments. Instead, inspired by audio spectrograms, we propose a harmonic overtone encoding scheme:

- Each non-drum program is mapped to a random but fixed "harmonic overtone series."
- Each played note generates K additional overtones (with decreasing velocity factors) at integer multiples of the original note's fundamental frequency, up to a maximum multiple of 5.

  For example: Piano programs could be mapped to the sequence $(2, 3, 5)$ with velocity factors $(0.6, 0.4, 0.1)$. If any piano track plays a note with fundamental frequency f_0 and velocity v, three additional notes at frequencies $(2f_0, 3f_0, 5f_0)$ are generated, quantized to the closest MIDI note, and added to the piano roll with onset velocities $(0.6v, 0.4v, 0.1v)$.
- We apply a linear amplitude decay to each generated overtone over the note's duration, scaling its velocity from full strength at note onset down to zero at the note's end. This helps distinguish overtones from actual note-onset events.
- The overtone-based encodings are assigned to a dedicated input channel, separate from the primary piano roll representation.

3.3 Model Architecture

CNNs have achieved state-of-the-art results across various MIR tasks, including section boundary detection for audio [21], onset detection [28], and beat tracking [29]. Given their success—particularly for the closely related task of audio-based section boundary detection—we adopt a CNN-based approach for our symbolic music task.

Recent work in MIR has demonstrated the effectiveness of finetuning CNNs pretrained on computer vision tasks for music-related applications [30, 31]. Inspired by these findings, we use MobileNetV3 [32], a lightweight

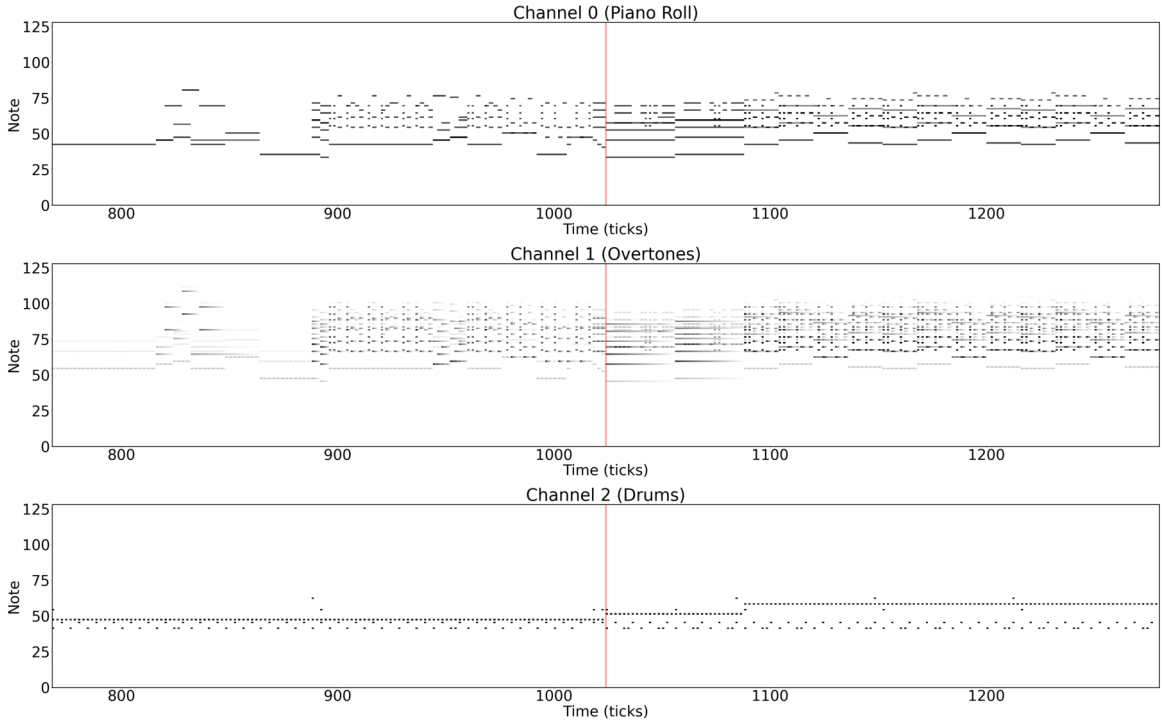


Figure 1. A patch from file ca05cc474fd2010484c1201bf57b3cfd from the training set. We overlay the vertical red line in the center of the patch in this figure to indicate that this is a positive (section boundary = True) training example.

yet effective CNN architecture originally designed for efficient image recognition. We train the model starting from weights pretrained on the ImageNet dataset [33]. We demonstrate in Section 5 that this approach yields slightly better results than training the same architecture from scratch.

We adopt the learning task outlined in [19–21], where the network aims to predict whether a section boundary is present at the center of the input patch. An example of an input to our neural network is given in Figure 1.

4. DATASET

The Lakh MIDI dataset (LMD) [9, 10] is a dataset of approximately 170k MIDI files widely used by the research community. In this section, we introduce a new subset of the LMD, which we call the Segmented Lakh MIDI Subset (SLMS).

While exploring the LMD, we noticed that thousands of the files contain MIDI markers. The "marker" event in MIDI is a meta event with a text string field that can be placed at any time location in a MIDI file. In some of these files, the markers are intended by the MIDI file authors to be section boundary markers, and in some of these, we found these markers to serve as reasonable segmentations. The authors of [21] point out that there can be a wide range of opinion in how to segment a piece of music—evaluating the section-level dual annotations in SALAMI against each other with an evaluation tolerance of 0.5 seconds, they found an F1 score of 74%. Therefore, we judge annotations to be segmentations whenever we find them to be reasonable, rather than whether they agree with how we would have segmented the file.

As far as we know, the existence of files containing annotated segmentations within the LMD has gone unnoticed by other researchers until now. Indeed, in the thesis in which the LMD was introduced, Raffel [9] found no files containing structural annotations, possibly because he searched for "text" events rather than "marker" events.

We began by deduping the LMD using the method in [34], which uses silence removal and quantized note onset chromagrams to identify when two files contain essentially the same musical information. To filter files that are unlikely to include valid segmentations, we then excluded files that had fewer than 3 markers and files that had an unreasonably low (less than 6) or high (more than 24) ratio of measures to markers. We also excluded files which had no markers between the first and last note onsets.

We then found a single MIDI file author (Benjamin Robert Tubb) whose name appeared in a majority (about 57%) of the files with markers. Tubb sequenced primarily 19th century popular songs, and files bearing his name have a distinctive layout. We therefore present the SLMS as two non-overlapping subsets: The *Tubb files* and the *non-Tubb files*. The non-Tubb files are more diverse in terms of both annotation style and musical style, and consist of styles including, but not limited to, rock, metal, jazz, solo classical piano, symphonic, and karaoke music. We also identified 5 files without Tubb's name in them that we believe he sequenced. We include them in the Tubb files.

This left us with 4466 Tubb files and 3336 non-Tubb files. We then performed a manual inspection of each of these files. We visualized each MIDI file in a digital au-

Dataset	# Songs	# Hours
SLMS (Tubb)	**3907**	**225.1**
SLMS (non-Tubb)	**2227**	**143.5**
SALAMI	1359	105.8
Harmonix	912	56.1
RWC	315	23.5

Table 1. Some annotated multi-track music datasets

dio workstation (DAW) to see if the markers appeared to represent a valid segmentation, and when we were unsure, we listened. We selected only files where all perceived segment boundaries occurred at bar lines (as defined by the time signature information within the files). After removing files containing clear errors (such as missing markers or misplaced markers) or markers that are not intended to serve as segment boundary markers, we were left with 3907 Tubb files (12.5% rejection rate) and 2227 non-Tubb files (33.2% rejection rate). We quantized all markers to bar lines and extracted all resulting marker information from these files, both in terms of seconds and in terms of beats elapsed since the start of the file. We release this information as the SLMS, including our train/validation/test split. [2]

This release constitutes the largest collection of human-segmented multi-track music of which we are aware. See Table 1 for information about how our dataset compares to other datasets of human-segmented music. We mention that the section boundary markers in many of these files also contain structural information (e.g., "verse", "chorus") in their text labels, which we have also extracted and included in the SLMS. This information may be useful for future work in Music Structure Analysis.

Despite the larger size of our dataset, we also acknowledge some shortcomings of our dataset relative to others. For example, SALAMI was created with a style guide for annotators, and contains multi-level annotations. Our dataset has neither property. Also, a majority of the files in SALAMI have two human annotations, while each of our files has only one—the annotation of the original MIDI file author.

Aside from one programmatic correction we discuss below, while creating our dataset, we resisted the urge to correct segmentations that we found to be incorrect. Instead, we chose simply to exclude files containing incorrect segmentations from the SLMS. We wanted our data contribution in this work to be primarily a record of what is already present in the LMD, rather than our subjective corrections to that data. We note that we excluded many files due to simple errors or omissions, and this provides an opportunity in future work to obtain a large amount of additional training data at the expense of some additional data correction effort. We release our starting list of 7802 files along with our final list of 6134 curated files in case other authors wish to carry out this work.

We made one programmatic correction to the Tubb files. Tubb often split measures at section boundaries containing pickups into two measures (e.g., a measure of 6/8 with a pickup beginning at the fifth eighth note would be split into a measure of 5/8 and a measure of 1/8, with the segment marker placed at the start of the 1/8 measure, rather than at the start of the next measure). For the Tubb files in our dataset, we moved markers forward to the start of the next measure when they occurred at a measure of less than half note duration followed by a measure with greater than or equal to a half note duration. Based on our experience looking at and listening to the files, we did not find this correction to introduce any segmentation errors. We note that we changed only the boundary marker locations, not the measure annotations themselves. There are many examples of short odd-time-signature measures near segment boundaries in our non-Tubb files, indicating that our model needs to be able to handle music with such embedded measure annotations to generalize to unseen data.

[2] https://github.com/m-malandro/SLMS

5. EXPERIMENTS

For all experiments, we train a MobileNetV3 [32] architecture using a binary cross-entropy loss function and optimize using AdamW [35] with a learning rate of 10^{-3} and weight decay of 10^{-2}. We apply early stopping when no improvement in validation F_1 score is observed for 5 consecutive epochs.

We use 5359 songs for training—3425 of the Tubb files and 1934 of the non-Tubb files. We use 246 Tubb and 100 non-Tubb songs for validation. Hence, our test set contains 236 Tubb and 193 non-Tubb songs. To ensure consistency in both training and evaluation, we exclude boundaries that occur near the beginning or end of a piece. Specifically, we disregard all segment boundaries that fall within 16 bars of the first or last note onset. While prior work (e.g., [19–21]) handles edge cases by padding the input with half a patch window, we observed inconsistencies in segment annotations near the beginnings and endings of some pieces in our dataset (e.g., whether the beginning of the final measure is marked as a segment boundary) and therefore instead adopt this boundary exclusion strategy, applying it uniformly to our method and all baselines. Hence, this paper focuses on identifying section boundaries in the "middle" of songs. Inconsistent boundary annotations near the beginnings and ends of songs in our dataset can be addressed in future work.

5.1 Our Method

We apply our piano roll-based method from Section 3 to our dataset, using $K = 3$ overtones per note. Since section boundary events in our dataset are relatively sparse, we include each positive example twice in each training epoch, while negative examples are included only once per epoch.

Since we are interested in barwise section boundary classification and have access to ground-truth measure positions via the MIDI files, we evaluate our approach using a per-measure hit rate—for each measure in each MIDI file, we create an input patch centered at that measure and ask the network to decide whether there is a section boundary

Model	F_1	Precision	Recall
Ours			
ensemble	**.7838**	.8001	.7682
our model	.7675	.7704	.7647
no pretraining	.7572	.7078	.8139
no overtones	.7593	.7140	.8108
no overtones, no drum split	.7661	.7572	.7753
Analogous audio			
per-measure	.5135	.6728	.4152
0.5s tolerance	.5523	.5456	.5590
CBM [36] (audio)			
per-measure	.4583	.4288	.4923
0.5s tolerance	.4488	.4414	.4564

Table 2. Primary test results. F_1 computed from section boundaries aggregated across Tubb and non-Tubb files.

there. We compute precision, recall, and F_1 score accordingly.

For our approach, we consider a model's prediction to be positive when the predicted probability exceeds a fixed threshold, which we set to 0.5 for all of our model variants. We experimented with more sophisticated post-processing techniques (including the peak-picking method of [19]), but found that they did not improve the F_1 score in our setting.

5.2 Ablation

To isolate the contributions of individual components of our approach, we train four model variants in an incremental ablation setup. Our final model is initialized with pretrained weights (Section 3.3) from MobileNetV3 [32], and uses both overtone encoding (Section 3.2) and drum track separation in the input representation. For the ablations, we experiment with omitting only pretraining, only overtones, and both overtones and drum separation. Additionally, we combined all four variants into a single bagged ensemble, averaging their output probabilities at inference time.

5.3 Audio-based Approaches

To compare our method with audio-based methods, we render the MIDI files in our dataset to audio using FluidSynth [37] and the Arachno soundfont [38].

In addition to evaluating the following audio-based approaches on a per-measure F_1 basis, following the evaluation procedures of the Boundary Retrieval task in the Music Information Retrieval Evaluation eXchange (MIREX), [3] we also evaluate with F_1 scores with tolerances of ± 0.5 seconds and ± 3 seconds.

5.3.1 Supervised Audio Baseline

For the first audio-based baseline, we implement an approach that is analogous to our MIDI-based approach and is similar to the approach in [19–21], replacing the piano rolls in our model inputs with synthesized audio. As in [19–21], we extract mel-scaled magnitude spectrograms from the synthesized audio using the same parameters. As in our symbolic approach, we separate harmonic and percussive content into distinct channels. Instead of applying HPSS as in [21], we render drums separately from the other instruments in each file, giving us the cleanest possible separation.

As with our models, we train a pretrained MobileNetV3 [32] on these inputs. We attempted to train the model only on patches centered on measure boundaries, as we did for our model, but the training did not converge. Hence, we trained on patches centered on all time frames of the input spectrogram as in [19–21]. In [19], the probability of sampling a positive example during training was increased by a factor of three. We found this to hurt the model's performance in our case, so we omit this aspect in our implementation. The remainder of the pipeline—including input resolution, training setup, and the proposed peak-picking method for boundary extraction—is kept identical to [19], with the exception of model bagging, which we also omit. Therefore, a fair comparison is between this model and any of our individual models.

For evaluation on a per-measure basis, we provide the trained model only with inputs centered at measure boundaries. In this setting, we evaluated both thresholding and peak-picking, and found peak-picking to perform best on the non-Tubb files in our validation set. Hence, we use peak-picking to post-process the outputs of this model for all reported results.

5.3.2 Unsupervised Audio Baseline

For our second audio-based baseline we use the correlation block-matching (CBM) segmentation algorithm [36], which is competitive with [21] on the RWC Pop dataset [6] and marginally worse than [21] on SALAMI [4].

The CBM algorithm requires two parameters: the number of bands n and the penalty weight w to the modulo-8 penalty function. The CBM algorithm also requires as input the list of bar onset times, which we provide as extracted from our MIDI files. We performed a grid search with $n \in \{7, 15\}$ and $w \in \{0, 0.04, 0.25, 0.375, 0.5, 0.75, 1\}$ and found that $n = 15, w = 0.25$ works best on the non-Tubb files in our validation set. We therefore apply the CBM algorithm with these parameters to the rendered audio of our test set. We also apply the CBM algorithm to our test data without providing the list of bar onset times, instead using the default bar-detection algorithm from their code (specifically, the downbeat estimator from the `madmom` toolbox [39] together with the bar tracking model from [40]).

5.4 Results and Discussion

An overview of results is given in Table 2, with a more detailed breakdown between the Tubb and non-Tubb files presented in Table 3. In these tables, "Analogous audio" refers to the supervised audio baseline described in

[3] https://www.music-ir.org/mirex/wiki/MIREX_HOME accessed 27 Mar 2025

Non-Tubb files			
Model	F_1	**Precision**	**Recall**
Ours			
ensemble	**.7160**	.7320	.7007
our model	.6981	.7015	.6947
no pretraining	.6974	.6413	.7644
no overtones	.6893	.6415	.7449
no overtones, no drum split	.6905	.6879	.6931
Analogous audio			
per-measure	.4435	.6729	.3309
1 bar tolerance	.4635	.7031	.3457
0.5s tolerance	.4466	.4440	.4493
3s tolerance	.6274	.6207	.6342
CBM [36] (audio)			
per-measure	.5436	.4994	.5962
1 bar tolerance	.6525	.6001	.7150
0.5s tolerance	.4856	.4634	.5101
3s tolerance	.6290	.6010	.6597

Tubb files			
Model	F_1	**Precision**	**Recall**
Ours			
ensemble	**.8559**	.8722	.8401
our model	.8413	.8434	.8393
no pretraining	.8234	.7844	.8665
no overtones	.8358	.7951	.8809
no overtones, no drum split	.8457	.8288	.8633
Analogous audio			
per-measure	.5678	.6728	.4912
1 bar tolerance	.7730	.9159	.6687
0.5s tolerance	.6424	.6313	.6538
3s tolerance	.7911	.7712	.8120
CBM [36] (audio)			
per-measure	.3718	.3544	.3911
1 bar tolerance	.6628	.6321	.6966
0.5s tolerance	.4105	.4171	.4041
3s tolerance	.6919	.7040	.6802

Table 3. Breakdown of test results between Tubb and non-Tubb files.

Section 5.3.1, and "CBM" refers to the unsupervised audio baseline described in Section 5.3.2.

Among our ablations, removing either overtone encoding or pretraining results in slight drops in performance. Interestingly, the variant without both overtone encoding and drum separation performs only marginally worse than the full model, suggesting that the core piano roll representation already provides a strong foundation. Performance on the Tubb files is higher than on the non-Tubb files for our models. As discussed in Section 4, the relative style homogeneity of the Tubb subset, as well as its higher representation in the training set, likely contribute to these results. Overall, these results indicate that while each component contributes incrementally to performance, even the simpler variants of our approach outperform strong baselines. Moreover, as in [19], ensemble averaging provides a practical and effective strategy to boost performance.

The fairest comparison between our method and the audio-based baselines is on the non-Tubb files in our test set, as these represent a wide range of musical genres and annotation styles, and therefore likely better represent generalizability to unseen data. As a secondary comparison, we compare results on the Tubb files in our test set as well.

Our model outperforms both audio-based baselines on both the non-Tubb and Tubb files in our test set. On the non-Tubb files, the CBM algorithm outperforms the supervised audio approach, with F_1 scores of 0.5436 and 0.4435, respectively. This F_1 score obtained by the CBM algorithm is in-line with the results obtained by its authors in [36], between their results (with a 0.5 second tolerance) on the SALAMI (0.42) and RWC Pop (0.64) datasets. When not supplying bar onset times to the CBM algorithm, performance was evaluated using 0.5 second and 3 second tolerances, and is similar to the performance obtained by supplying the bar onset times. On the Tubb files, the supervised audio baseline outperforms the CBM algorithm, with F_1 scores of 0.5678 and 0.3718, respectively. The performance of our approach was considerably higher, with an F_1 score of 0.7675 on the non-Tubb files and 0.8413 on the Tubb files in our test set.

We note that even if we apply loose tolerances to the outputs of our audio-based baselines (specifically, 1-bar or 3-second tolerances), the resulting F_1 scores are still below those achieved by our approach with strict tolerance.

6. CONCLUSION AND FUTURE WORK

We have introduced a new symbolic music dataset (the SLMS) for Music Structure Analysis, containing 6134 human-annotated MIDI files. We extracted and manually curated this dataset from the Lakh MIDI dataset. We used this dataset to train a CNN to detect section boundaries in symbolic music, and have shown that our network outperforms both the analogous audio-based learning approach and the competitive correlation block-matching segmentation algorithm.

Our work was based on adapting the audio-based methods in [19–21] to MIDI data. The ideas from the audio-based approach in [21] that we have not yet explored are the use of two-level annotations (which are not present in our dataset) and the use of SSLMs as additional inputs to the model [20]. Based on our results in this work and the improvement in [21] over previous audio-based approaches, we do not expect that incorporating SSLMs into the audio-based supervised approach implemented in this paper would close the wide gap between its performance and the performance of our models. However, developing MIDI-based SSLMs may improve the performance of our models. In future work, we plan to clean and expand the SLMS via manual corrections, explore alternative model architectures, and explore the use of MIDI-based SSLMs as model inputs.

7. REFERENCES

[1] S. Dai, H. Zhang, and R. B. Dannenberg, "Automatic Analysis and Influence of Hierarchical Structure on Melody, Rhythm and Harmony in Popular Music," in *Proceedings of the 2020 Joint Conference on AI Music Creativity (CSMC-MuMe)*, 2020.

[2] S. Bassan, Y. Adi, and J. Rosenschein, "Unsupervised Symbolic Music Segmentation using Ensemble Temporal Prediction Errors," in *Interspeech*, 2022, pp. 2423–2427.

[3] O. Nieto, G. J. Mysore, C.-i. Wang, J. B. L. Smith, J. Schlüter, T. Grill, and B. McFee, "Audio-Based Music Structure Analysis: Current Trends, Open Challenges, and Applications," *Transactions of the International Society for Music Information Retrieval*, Dec 2020.

[4] J. B. L. Smith, J. A. Burgoyne, I. Fujinaga, D. D. Roure, and J. S. Downie, "Design and Creation of a Large-Scale Database of Structural Annotations," in *Proc. 12th Int. Society for Music Information Retrieval Conf.*, Miami, Florida, USA, 2011, pp. 555–560.

[5] O. Nieto, M. McCallum, M. Davies, A. Robertson, A. Stark, and E. Egozy, "The Harmonix Set: Beats, Downbeats, and Functional Segment Annotations of Western Popular Music," in *Proc. 20th Int. Society for Music Information Retrieval Conf.*, 2019.

[6] M. Goto, H. Hashiguchi, T. Nishimura, and R. Oka, "RWC Music Database: Popular, Classical and Jazz Music Databases," in *Proc. 3rd Int. Conf. on Music Information Retrieval*. ISMIR, 2002.

[7] ——, "RWC Music Database: Music Database: Music Genre Database and Musical Instrument Sound Database," in *Proc. 4th Int. Conf. on Music Information Retrieval*. ISMIR, 2003.

[8] M. Goto, "AIST Annotation for the RWC Music Database," in *Proc. 7th Int. Conf. on Music Information Retrieval*. ISMIR, 2006, pp. 359–360.

[9] C. Raffel, "Learning-Based Methods for Comparing Sequences, with Applications to Audio-to-MIDI Alignment and Matching," Ph.D. dissertation, 2016.

[10] ——, "The Lakh MIDI Dataset v0.1," https://colinraffel.com/projects/lmd/.

[11] S. Dai, H. Yu, and R. B. Dannenberg, "What is missing in deep music generation? A study of repetition and structure in popular music," in *Proc. of the 23rd Int. Society for Music Information Retrieval Conf.*, Bengaluru, India, 2022.

[12] L. Casini and B. L. T. Sturm, "Tradformer: A Transformer Model of Traditional Music Transcriptions," in *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*, L. D. Raedt, Ed. International Joint Conferences on Artificial Intelligence Organization, 2022, pp. 4915–4920, AI and Arts. [Online]. Available: https://doi.org/10.24963/ijcai.2022/681

[13] S. Mossmyr, E. Hallström, B. L. Sturm, V. H. Vegeborn, and J. Wedin, "From Jigs and Reels to Schottisar och Polskor: Generating Scandinavian-like Folk Music with Deep Recurrent Networks," in *16th Sound and Music Computing Conference (SMC2019)*, 2019.

[14] H. Chen, J. B. L. Smith, J. Spijkervet, J.-C. Wang, P. Zou, B. Li, Q. Kong, and X. Du, "SymPAC: Scalable Symbolic Music Generation With Prompts And Constraints," in *Proc. 25th Int. Society for Music Information Retrieval Conf.*, San Francisco, CA, United States, 2024, pp. 1029–1036.

[15] J. Foote, "Automatic Audio Segmentation Using a Measure of Audio Novelty," in *2000 IEEE International Conference on Multimedia and Expo. ICME2000. Proceedings. Latest Advances in the Fast Changing World of Multimedia (Cat. No.00TH8532)*, vol. 1, 2000, pp. 452–455.

[16] Z. Wang*, K. Chen*, J. Jiang, Y. Zhang, M. Xu, S. Dai, G. Bin, and G. Xia, "POP909: A Pop-Song Dataset for Music Arrangement Generation," in *Proc. 21st Int. Society for Music Information Retrieval Conf.*, 2020.

[17] H. Schaffrath. (Accessed: 9 Mar 2025) The Essen Folksong Collection. [Online]. Available: https://kern.humdrum.org/cgi-bin/browse?l=/essen

[18] Z.-S. Lin, Y.-C. Kuo, T.-Y. Hung, W.-Y. Lin, Y.-H. CHU, T.-K. Wang, J.-H. Huang, C. Chang, C. Julio, G. Hsieh, and L. Su, "S3: A Symbolic Music Dataset for Computational Music Analysis of Symphonies," in *Extended Abstracts for the Late-Breaking Demo Session of the 25th Int. Society for Music Information Retrieval Conf.*, San Francisco, CA, USA, 2024.

[19] K. Ullrich, J. Schlüter, and T. Grill, "Boundary Detection in Music Structure Analysis using Convolutional Neural Networks," in *15th Int. Soc. Music Information Retrieval Conf.*, 2014.

[20] T. Grill and J. Schlüter, "Music boundary detection using neural networks on spectrograms and self-similarity lag matrices," in *2015 23rd European Signal Processing Conference (EUSIPCO)*, 2015, pp. 1296–1300.

[21] ——, "Music Boundary Detection Using Neural Networks on Combined Features and Two-Level Annotations," in *Proc. 16th Int. Society for Music Information Retrieval Conf.*, 2015, pp. 531–537.

[22] M. C. McCallum, "Unsupervised Learning of Deep Features for Music Segmentation," in *ICASSP 2019 - 2019 IEEE International Conference on Acoustics,*

Speech and Signal Processing (ICASSP), 2019, pp. 346–350.

[23] M. Buisson, B. Mcfee, S. Essid, and H.-C. Crayencour, "Learning Multi-Level Representations for Hierarchical Music Structure Analysis," in *Proceedings of ISMIR 2022*, Bengaluru, India, Dec. 2022. [Online]. Available: https://hal.science/hal-03780032

[24] M. Buisson, B. McFee, S. Essid, and H. C. Crayencour, "Self-supervised learning of multi-level audio representations for music segmentation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 32, pp. 2141–2152, 2024.

[25] M. Buisson, C. Ick, T. Xi, and B. McFee, "Zero-Shot Structure Labeling with Audio And Language Model Embeddings," in *Extended Abstracts for the Late-Breaking Demo Session of the 25th International Society for Music Information Retrieval Conference (ISMIR)*, San Francisco, CA, United States, Nov. 2024. [Online]. Available: https://hal.science/hal-04764247

[26] J.-C. Wang, Y.-N. Hung, and J. B. L. Smith, "To catch a chorus, verse, intro, or anything else: Analyzing a song with structural functions," in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 416–420.

[27] T. Kim and J. Nam, "All-in-one metrical and functional structure analysis with neighborhood attentions on demixed audio," in *2023 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA)*, 2023, pp. 1–5.

[28] J. Schlüter and S. Böck, "Improved musical onset detection with convolutional neural networks," in *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2014, pp. 6979–6983.

[29] E. P. MatthewDavies and S. Böck, "Temporal convolutional networks for musical audio beat tracking," in *2019 27th European Signal Processing Conference (EUSIPCO)*, 2019, pp. 1–5.

[30] S. Lattner, "SampleMatch: Drum Sample Retrieval by Musical Context," in *Proc. of the 23rd Int. Society for Music Information Retrieval Conf.*, Bengaluru, India, 2022, pp. 781–788.

[31] G. Argüello, L. A. Lanzendörfer, and R. Wattenhofer, "Cue Point Estimation using Object Detection," in *Proc. of the 25th Int. Society for Music Information Retrieval Conf.*, 2024, pp. 405–412.

[32] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan *et al.*, "Searching for mobilenetv3," in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 1314–1324.

[33] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255.

[34] M. Malandro, "Composer's Assistant: An Interactive Transformer for Multi-Track MIDI Infilling," in *Proc. 24th Int. Society for Music Information Retrieval Conf.*, Milan, Italy, 2023, pp. 327–334.

[35] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=Bkg6RiCqY7

[36] A. Marmoret, J. E. Cohen, and F. Bimbot, "Barwise Music Structure Analysis with the Correlation Block-Matching Segmentation Algorithm," *Transactions of the International Society for Music Information Retrieval*, Nov 2023.

[37] "FluidSynth," accessed: 23 Mar 2025. [Online]. Available: https://www.fluidsynth.org/

[38] "Arachno SoundFont," accessed: 23 Mar 2025. [Online]. Available: https://www.arachnosoft.com/main/download.php?id=soundfont-sf2

[39] S. Böck, F. Korzeniowski, J. Schlüter, F. Krebs, and G. Widmer, "madmom: a new Python Audio and Music Signal Processing Library," in *Proceedings of the 24th ACM International Conference on Multimedia*, Amsterdam, The Netherlands, 10 2016, pp. 1174–1178.

[40] F. Krebs, S. Böck, and G. Widmer, "An Efficient State-Space Model for Joint Tempo and Meter Tracking," in *16th Int. Soc. for Music Information Retrieval Conf.*, 2015.